

Objektorientiertes Programmieren

Übungsbeispiele

1 String Klasse

Entwerfen und implementieren Sie eine **String** Klasse in C++. Ziel ist es, ein grundlegendes Verständnis für dynamische Speicherverwaltung, Kopiersemantik und Operatorüberladung zu entwickeln.

Folgende Punkte müssen dabei beachtet werden:

- Teilen Sie Ihren Code in Header und CPP Dateien auf.
- Implementieren Sie Ihre String Klasse in einen Namespace (Bezeichnung können Sie selber wählen).
- Testen Sie Ihren Code ausgiebig und berücksichtigen Sie Randbedingungen.
- Achten Sie auf Ihr Speichermanagment.

Stufe 1

Folgende Elemente sollen als Klassen implementiert werden:

- Implementieren Sie einen **Konstruktor** für die Klasse **String**, der einen C-String (const char*) als Parameter erhält.
 - die Länge des übergebenen C-Strings bestimmen - die Länge kann zusätzlich in der String Klasse als Membervariable gespeichert werden.
 - dynamisch Speicher für die Zeichenkette inklusive Nullterminator reservieren
 - den Inhalt des C-Strings in den internen Speicher kopieren (**char***)
 - Es soll auch möglich sein ein String Objekt leer zu initialisieren bzw. mit einem nullptr.
- Implementieren Sie einen **Destruktor** für die Klasse **String**, der den intern verwendeten dynamisch allozierten Speicher korrekt freigibt.
 - den zuvor reservierten Speicher des internen char-Arrays freigeben
 - sicherstellen, dass keine Speicherlecks entstehen

- nach der Freigabe einen gültigen, definierten Objektzustand hinterlassen
- Implementieren Sie die Methode **getString()**, die einen Zeiger auf den intern C-String zurückgibt.
- Implementieren Sie die Methode **getLength()**, welche die Länge der Zeichenkette zurückgibt.

Stufe 2

- Implementieren Sie einen **Kopierkonstruktor**, der ein bestehendes String-Objekt tief kopiert.

Der Kopierkonstruktor soll:

- neuen dynamischen Speicher in passender Größe reservieren
- den Inhalt des Quell-Strings vollständig kopieren
- die Länge korrekt übernehmen
- sicherstellen, dass Quelle und Kopie unabhängig voneinander verwaltet werden
- Implementieren Sie den **Kopierzuzuweisungsoperator** (operator=), der einem bestehenden String-Objekt den Inhalt eines anderen zuweist.

Der Operator soll:

- Selbstzuweisung erkennen und korrekt behandeln
- alten Speicher sicher freigeben
- neuen Speicher reservieren und den Inhalt kopieren
- Länge und internen Zustand korrekt aktualisieren
- Implementieren Sie einen **Move-Konstruktor**, der die Ressourcen eines temporären oder zu verschiebenden String-Objekts übernimmt.

Der Move-Konstruktor soll:

- den internen Speicherzeiger und die Länge vom Quellobjekt übernehmen
- das Quellobjekt in einen gültigen, leeren Zustand versetzen
- keine Kopie der Daten anlegen
- Implementieren Sie den **Move-Zuweisungsoperator** (operator=), der die Ressourcen eines anderen String-Objekts übernimmt.

Der Move-Zuweisungsoperator soll:

- Selbstzuweisung erkennen
- den eigenen alten Speicher freigeben
- Speicherzeiger und Länge vom Quellobjekt übernehmen
- das Quellobjekt in einen definierten, leeren Zustand zurücksetzen
- Die Move-Zuweisung soll effizient sein und keine unnötigen Kopien erzeugen.

Stufe 3

Ergänzen Sie Ihre String Klasse mit folgenden nützlichen Methoden & Operatoren

- concatenate: für andere Objekte von String und für const char*
- operator+ für andere Objekte von String und für const char*
- operator+= für andere Objekte von String und für const char*
- conversion function zu einem char*

Bewertung

Aspekt	Bewertung
Speichermanagement	10%
Stufe 1	30%
Stufe 2	40%
Stufe 3	20%